

APK AI Analyzer

План продукта и разработки

Локальный анализ APK

Объяснимая оценка риска

Мультипровайдерные AI-отчёты

APK

ФАКТЫ

РИСК

ОТЧЁТ

Содержание

Полный пользовательский план продукта: от сценария анализа APK до AI-интеграции, критериев готовности и дальнейшего развития.

1. Что именно мы создаём	2
2. Как это будет работать для пользователя	2
3. Что войдёт в первую рабочую версию	3
Обязательно	3
4. Технологический стек	3
5. Экраны приложения	3
6. Правильная логика оценки риска	4
7. Как будет использоваться AI	4
Выбор AI, модели и API-ключи	4
Специализированные AI-запросы	5
8. План разработки по этапам	5
Этап 1. Прототип локального анализа	5
Этап 2. Движок правил и оценка риска	5
Этап 3. Интерфейс и отчёты	6
Этап 4. AI-объяснение	6
Этап 5. Установленные приложения	6
9. Критерии готовности MVP	7
10. Развитие после MVP	7

1. Что именно мы создаём

APK AI Analyzer - приложение для Android, в котором пользователь выбирает APK-файл, после чего приложение безопасно исследует его, не устанавливая и не запуская. Оно показывает уровень риска, объясняет конкретные подозрительные признаки простыми словами и формирует отчёт.

2. Как это будет работать для пользователя

1. Пользователь нажимает «**Выбрать APK**» и выбирает скачанный файл через системное окно Android.
2. Приложение копирует файл во временную закрытую область и проверяет, что это действительно корректный APK. Файл не устанавливается и не запускается.
3. Локальный анализатор считывает SHA-256 и извлекает:
 - имя пакета и версию;
 - минимальную и целевую версии Android;
 - запрошенные разрешения;
 - activities, services, receivers и providers;
 - экспортируемые компоненты и intent-фильтры;
 - сведения о цифровой подписи;
 - подозрительные URL, домены, IP-адреса и строки;
 - использование потенциально опасных API и признаки упаковщиков/обфускации;
 - встроенные нативные библиотеки.
4. Движок правил сопоставляет признаки между собой. Например, одно разрешение на SMS ещё ничего не доказывает, но сочетание чтения SMS, автозапуска, скрытого сервиса и отправки данных во внешний домен заметно повышает риск.
5. Приложение выдаёт один из результатов:
 - **низкий риск** - явных опасных комбинаций не найдено;
 - **нужно внимание** - есть признаки, требующие проверки;
 - **высокий риск** - обнаружено несколько сильных взаимосвязанных индикаторов;
 - **недостаточно данных** - файл повреждён, защищён упаковщиком или анализ неполон.
6. Стандартный режим - **AI-объяснение по фактам и фрагментам**. После первого явного согласия во внешний сервис отправляется не целый APK и не случайный бинарный кусок, а небольшой JSON: локальная оценка риска, сработавшие правила, связанные разрешения/API и выбранные анализатором короткие подозрительные фрагменты строк. Перед первой отправкой пользователь видит состав данных; постоянный предварительный просмотр можно включить в настройках.
7. В конце создаётся отчёт: краткий итог, уровень риска, доказательства, ограничения анализа и рекомендуемые действия.
8. Пользователь может удалить выбранный APK-файл, открыть системное окно удаления установленного приложения, перейти к его настройкам или поделиться отчётом.

3. Что войдёт в первую рабочую версию

Обязательно

- выбор одного .apk через системный файловый диалог;
- полностью локальный разбор файла без его запуска;
- SHA-256, сведения о пакете, SDK и подписи;
- анализ разрешений и компонентов манифеста;
- извлечение ограниченного набора строк, URL, доменов и IP;
- версия правил и объяснимый риск-скоринг;
- экран результата с доказательствами;
- стандартный AI-режим «факты + очищенные фрагменты», активируемый только после явного согласия пользователя;
- выбор AI-провайдера, модели и собственного API-ключа;
- история анализов на устройстве;
- экспорт отчёта в PDF и JSON;
- удаление выбранного файла, если Android это разрешает;
- защита от слишком больших файлов, zip-bomb и повреждённых архивов.

4. Технологический стек

- язык: **Kotlin**;
- интерфейс: **Jetpack Compose + Material 3**;
- архитектура: разделение UI, сценариев, анализатора и хранения данных;
- асинхронность: Kotlin Coroutines и Flow;
- локальная история: Room;
- настройки и согласия: DataStore;
- фоновые ограниченные задачи: WorkManager;
- сеть: Retrofit/OkHttp или Ktor Client;
- внедрение зависимостей: Hilt;
- отчёты: Android PdfDocument для PDF и отдельный JSON-формат доказательств;
- единый AI-слой с адаптерами для DeepSeek, OpenAI, Gemini и совместимых API;
- два режима подключения: собственный API-ключ пользователя напрямую или управляемый серверный шлюз в будущем.

5. Экраны приложения

- 1. Первый запуск** - что анализируется, что приложение не гарантирует, какие данные могут уйти в AI.
- 2. Главная** - «Выбрать APK», «Установленные приложения» (позже), история.
- 3. Подтверждение анализа** - имя, размер, источник и выбранный AI-режим; по умолчанию «факты + очищенные фрагменты» после согласия.
- 4. Прогресс** - этапы проверки и возможность отмены.
- 5. Результат** - уровень риска, краткий вывод, главные индикаторы.
- 6. Доказательства** - каждое правило, источник факта и вес.

- 7. **AI-объяснение** - понятный текст, рекомендации и явная отметка, что это объяснение модели.
- 8. **Отчёт** - просмотр, экспорт PDF/JSON, поделиться.
- 9. **Действия** - удалить файл, удалить установленное приложение через Android, открыть сведения о приложении.
- 10. **Настройки** - AI-провайдер, модель, API-ключ, тест подключения, приватность, срок хранения истории и обновление правил.

6. Правильная логика оценки риска

Нельзя просто складывать все разрешения. Риск должен строиться из доказуемых комбинаций:

- отдельный индикатор получает категорию, вес, уверенность и ссылку на место, где он найден;
- правила группируются по поведению: SMS/звонки, слежение, сбор данных, скрытый автозапуск, загрузка кода, административный контроль, фишинг;
- сильные комбинации дают больший вклад, чем сумма одиночных слабых сигналов;
- легитимные контексты снижают вес, но не удаляют доказательство;
- итог содержит не только число, но и причины, ограничения и полноту анализа;
- версия правил записывается в отчёт, чтобы результат можно было воспроизвести.

Пример: INTERNET сам по себе почти ничего не значит. READ_SMS сам по себе требует внимания. Но READ_SMS + RECEIVE_BOOT_COMPLETED + фоновый service + неизвестный домен + попытка скрыть иконку - уже сильная связка.

7. Как будет использоваться AI

AI получает связанный структурированный набор фактов, например:

- идентификаторы сработавших правил;
- категории разрешений;
- типы экспортируемых компонентов;
- домены в нормализованном виде, если пользователь разрешил;
- уровень полноты анализа;
- числовую оценку движка и её расшифровку.

В рекомендуемом режиме к этим фактам добавляются **короткие подозрительные фрагменты**, выбранные локальным анализатором:

- строка с командой shell, URL или необычным путём;
- название чувствительного Android API и тип операции;
- короткий контекст вокруг подозрительного вызова из DEX;
- фрагмент настройки exported component или network security;
- причина, по которой анализатор выбрал этот фрагмент.

Выбор AI, модели и API-ключа

Пользователь сможет создать один или несколько AI-профилей. В каждом профиле указываются:

- провайдер: DeepSeek, OpenAI, Gemini или пользовательский OpenAI-совместимый API;
- API-ключ аккаунта выбранного провайдера;

- модель - из доступного списка либо её точный идентификатор вручную;
- для совместимого API - адрес сервера;
- кнопка «**Проверить подключение**» без отправки данных APK.

API-ключ относится к провайдеру, а модель выбирается отдельно. Ключ, введённый пользователем, не встраивается в APK, не попадает в отчёты и журналы и хранится на телефоне в зашифрованном виде с использованием Android Keystore. При выборе пользовательского сервера приложение перед отправкой ключа показывает точный адрес назначения.

По умолчанию один выбранный профиль используется для всех AI-функций. В расширенных настройках можно назначить разные модели для объяснения результата и составления текста отчёта. Приложение никогда не переключается на другого провайдера незаметно: резервная модель используется только после отдельного включения.

Специализированные AI-запросы

Вместо одного универсального промпта используются отдельные версионизируемые запросы:

- 1. Объяснение находок** - переводит правила, разрешения и API-контекст на понятный язык.
- 2. Предполагаемое поведение** - описывает только сценарии, подтверждённые переданными доказательствами, и отдельно отмечает неопределённость.
- 3. Рекомендации** - формирует приоритетные безопасные действия для пользователя.
- 4. Текст отчёта** - создаёт краткое резюме и разделы отчёта, не меняя локальную категорию риска и рассчитанный балл.

Каждый запрос требует структурированный JSON фиксированного формата. Если конкретная модель не поддерживает строгую схему, приложение проверяет обычный JSON и отклоняет некорректный ответ. Итоговый PDF/JSON собирает само приложение из локальных доказательств и проверенных AI-полей; модель не получает возможность самостоятельно переписать факты анализа.

8. План разработки по этапам

Этап 1. Прототип локального анализа

- создать Android-проект и базовую навигацию;
- сделать безопасный импорт APK;
- вычислять хеш и читать манифест/подпись;
- показать сырой список признаков;
- протестировать отмену, повреждённые и большие файлы.

Результат: APK выбирается и разбирается полностью офлайн.

Этап 2. Движок правил и оценка риска

- определить формат правил;
- реализовать веса, комбинации, исключения и полноту анализа;
- добавить набор первых проверок;
- создать тесты на каждое правило;
- откалибровать пороги на тестовом наборе.

Результат: воспроизводимый риск-скоринг с доказательствами.

Этап 3. Интерфейс и отчёты

- сделать понятные экраны результата;
- добавить историю в Room;
- экспортировать PDF/JSON;
- реализовать удаление выбранного файла и системные действия;
- добавить доступность, русский язык и корректные состояния ошибок.

Результат: полезный офлайн-MVP без LLM.

Этап 4. AI-объяснение

- создать единый AI-клиент и адаптеры DeepSeek, OpenAI, Gemini и совместимого API;
- добавить AI-профили: провайдер, модель, API-ключ, адрес сервера и проверка соединения;
- реализовать безопасное локальное хранение пользовательских ключей через Android Keystore;
- подготовить отдельные версионизируемые запросы для объяснения, поведения, рекомендаций и текста отчёта;
- подготовить строгую входную и выходную JSON-схемы;
- добавить согласие на передачу данных и предварительный просмотр отправляемых фактов;
- добавить выбор и очистку коротких подозрительных строк/API-контекста;
- реализовать тайм-ауты, лимиты, повтор и безопасный резервный сценарий;
- протестировать инъекции инструкций через строки внутри APK;
- явно отделить вывод движка от текста LLM.

Результат: AI понятным языком объясняет локальные доказательства, но не меняет оценку риска.

Этап 5. Установленные приложения

- добавить отдельный режим просмотра установленных пакетов;
- обработать package visibility и split APK;
- подготовить декларацию QUERY_ALL_PACKAGES, если публикация идёт через Google Play;
- добавить системное подтверждение удаления и переход к настройкам приложения.

Результат: пользователь может проверить доступные установленные приложения в рамках ограничений Android.

9. Критерии готовности MVP

- приложение анализирует корректный APK без его установки и без доступа к интернету;
- повреждённый или опасно сформированный архив не приводит к падению или переполнению памяти;
- каждый риск имеет конкретные доказательства;
- одинаковый файл и версия правил дают одинаковый локальный результат;
- без согласия пользователя наружу не отправляется ничего;
- AI недоступен - локальный отчёт всё равно полностью работает;
- в APK отсутствуют заранее встроенные ключи; пользовательский ключ хранится локально в зашифрованном виде и не попадает в отчёт или логи;
- пользователь может выбрать провайдера и модель, проверить подключение, заменить или удалить ключ;
- итоговый отчёт собирается приложением, а AI не может изменить локальные доказательства, категорию риска или рассчитанный балл;
- PDF и JSON содержат SHA-256, время, версию приложения/правил, выводы и ограничения;
- кнопки удаления всегда используют разрешённый Android сценарий с участием пользователя;
- на тестовом наборе измерены false positive, false negative и доля файлов с неполным анализом.

10. Развитие после MVP

- поддержка split APK и контейнеров .apks/ .apk/ .xapk;
- интеграция с VirusTotal по SHA-256: отправляется только хеш, и если такой APK уже известен сервису, приложение получает существующий отчёт без загрузки файла;
- общий интерфейс для других hash-reputation сервисов, чтобы позже подключать дополнительные источники независимо от основного анализатора;
- обновляемые подписанные наборы правил;
- локальная маленькая модель для офлайн-объяснений;
- наблюдение сетевых метаданных через добровольно включаемый VPNService без обещания расшифровки TLS;
- отдельная серверная песочница для динамического анализа - только как самостоятельный продуктовый модуль;
- экспорт в STIX/SARIF или формат корпоративной DFIR-системы.